Web VAPT Section 03 - Directory and File Enumeration

Professional documentation for Gobuster-based discovery of hidden paths, files, and application locations.

Enterprise Security Assessment Lab

PROJECT AREA	Web Application Vulnerability Assessment and Penetration Testing
PRIMARY TARGET	DVWA running in local lab environment
TESTING ENVIRONMENT	Kali Linux attack machine with Docker-based vulnerable target
PREPARED BY	Jagriti Banerjee
DOCUMENT TYPE	Individual Web VAPT Attack-Section PDF

AUTHORIZED LAB TESTING ONLY

No live client data. No production systems. Screenshots and outputs must be redacted before public upload.

1. Document Purpose

This individual PDF documents the Web VAPT attack section titled 'Directory and File Enumeration'. It is designed for the Enterprise Security Assessment Lab GitHub portfolio and describes the objective, tooling, commands, sanitized output, evidence mapping, security interpretation, remediation, and redaction requirements for this specific section.

> This document is written as a public portfolio version. It does not contain live client data, production system details, unredacted credentials, session cookies, raw hash dumps, or destructive attack instructions.

Field	Details
Project Area	Web Application Vulnerability Assessment and Penetration Testing
Primary Target	DVWA running in a local lab environment
Testing Environment	Kali Linux attack machine with Docker-based vulnerable target
Testing Style	Reconnaissance -> Enumeration -> Manual validation -> Exploitation proof -> Evidence capture
Methodology Reference	OWASP Web Security Testing methodology, OWASP Top 10, and PTES-style workflow
Prepared By	Jagriti Banerjee

2. Section Overview

Item	Details
Section Number	03
Section Title	Web VAPT Section 03 - Directory and File Enumeration
Risk / Severity Style	Medium / Informational in real-world context
OWASP Mapping	Supports A01 Broken Access Control, A05 Security Misconfiguration, and A06 Component Exposure review.
Primary Evidence Count	3

2.1 Objectives

- Discover hidden web paths that are not visible through normal navigation.
- Identify setup, login, security, vulnerability, and information disclosure pages.
- Document accessible application paths for later manual validation.
- Create enumeration evidence for the GitHub portfolio.

2.2 Tools Used

Tool	Purpose / Use in this section
Gobuster	Directory and file enumeration.
DIRB/SecLists wordlists	Used as part of the controlled Web VAPT workflow.
Browser	Manual verification and proof screenshots.
curl	HTTP header and endpoint testing.

3. Methodology and Execution Workflow

The execution workflow follows a controlled lab approach: confirm authorization and target scope, run only the tools required for the section, save outputs, validate observations manually where applicable, capture screenshots, redact sensitive values, and document findings without exaggeration.

3.1 Command Reference

```
gobuster dir   -u http://127.0.0.1:8081   -w /usr/share/wordlists/dirb/common.txt   -x
    php,html,txt,bak   -o tool-outputs/gobuster-directory-results.txt
```

3.2 Expected / Sanitized Output

```
Gobuster returned accessible application paths.
Directory and file enumeration identified pages for manual validation.
Results were saved as tool output and screenshot evidence.
```

3.3 Validation Criteria

- The target must be the local DVWA or authorized lab environment only.
- The output must match the intended section objective.
- Evidence screenshots must clearly show the validation result or tool output.
- Any sensitive values must be blurred, cropped, removed, or replaced with placeholders.
- The section must not claim findings that are not supported by evidence.

4. Evidence Mapping

Evidence File	Evidence Type / Reason
06-gobuster-directory-results.png	Directory discovery output.
06-gobuster-directory-results(M).png	Additional enumeration evidence.
Directory-&-File-Enumeration-with-Gobuster.png	Directory enumeration summary evidence.

5. Professional Security Analysis

Directory enumeration is often the bridge between basic recon and useful manual testing. It shows which application areas should be reviewed next.

In DVWA, many discovered paths are expected, but the process mirrors how testers identify hidden paths in a real assessment.

The screenshot evidence should be treated as enumeration proof, not as a standalone vulnerability unless sensitive or restricted paths are exposed.

5.1 Why This Section Matters

This section matters because professional VAPT is not only about running a tool. It requires a clear objective, controlled execution, evidence capture, correct interpretation, and practical remediation. For recruiters and reviewers, this PDF shows that the work was structured, documented, and aligned with real security methodology.

5.2 Common Mistakes to Avoid

- Uploading raw outputs that expose cookies, hashes, paths, or credentials.
- Claiming production-level impact without production context.
- Reporting scanner output as a confirmed vulnerability without validation.
- Using unclear screenshots that do not show the relevant result.
- Mixing unrelated phases in one evidence file without explanation.

6. Risk Analysis

- Backup files can reveal source code or credentials.
- Setup pages can allow misconfiguration or reset behaviour.
- Unprotected admin paths may enable unauthorized access.
- Debug pages can expose internal server information.

6.1 Real-World Impact Statement

The real-world impact depends on the affected asset, authentication context, data sensitivity, privilege level, exposed functionality, and compensating controls. Because this project uses DVWA in a lab, severity is described as a real-world context estimate rather than a formal client CVSS score.

7. Remediation and Hardening Guidance

- Remove unused files and backup copies.
- Disable setup/install pages after deployment.
- Protect sensitive paths with authentication and authorization.
- Disable directory listing.
- Review web roots for accidental files before release.

7.1 Verification After Remediation

1. Re-run the original test case in the lab or development environment.
2. Confirm that the previous vulnerable behaviour no longer appears.
3. Check server logs for blocked or rejected attempts.
4. Capture new evidence showing the fix or defensive control.
5. Update the report status from validated/open to fixed/retested if applicable.

8. GitHub Redaction and Publishing Checklist

> Cookies, hashes, credentials, session IDs, shell paths, database output, and private details must be redacted before public upload.

Item	Action Before Upload
PHPSESSID / Cookies	Blur or replace with <REDACTED_COOKIE>.
Password hashes	Blur or replace with <REDACTED_HASH>.

Item	Action Before Upload
Cracked passwords	Do not publish; replace with <REDACTED_PASSWORD>.
Database dumps	Crop, blur, or summarize instead of uploading raw data.
Webshell paths/files	Do not upload shell source files; redact paths if needed.
Local usernames/paths	Blur if private or unnecessary.
Tool raw outputs	Upload only sanitized outputs or summary documents.

8.1 Safe Git Commands Before Push

```
git status
git diff --cached
git diff --cached | grep -iE "password|secret|token|cookie|PHPSESSID|hash|credential"
```

9. Conclusion

This document completes the individual Web VAPT PDF for Section 03. It documents the section objective, execution workflow, evidence mapping, professional interpretation, risk, remediation, and GitHub safety checks. The content is aligned to the actual Enterprise Security Assessment Lab Web VAPT evidence set and avoids unsupported production claims.
